

# S-Agent

## Spatial Tool-Use Elicits Reasoning for Spatial Intelligence

Yalun Dai<sup>1,\*</sup> Hao Li<sup>1,\*</sup> Shulin Tian<sup>1</sup> Runmao Yao<sup>1</sup> Yuhao Dong<sup>1</sup>  
Fangzhou Hong<sup>1,\*</sup> Zhaoxi Chen<sup>1,\*</sup> Fangfu Liu<sup>2</sup> Baoliang Tian<sup>3</sup> Tao Wang<sup>3,†</sup>  
Dingwen Zhang<sup>4</sup> Kim-Hui Yap<sup>1,†</sup> Ziwei Liu<sup>1,\*</sup>

<sup>1</sup>NTU <sup>2</sup>THU <sup>3</sup>ByteDance <sup>4</sup>USTC \*Ropedia

\*Equal contribution. †Corresponding author.

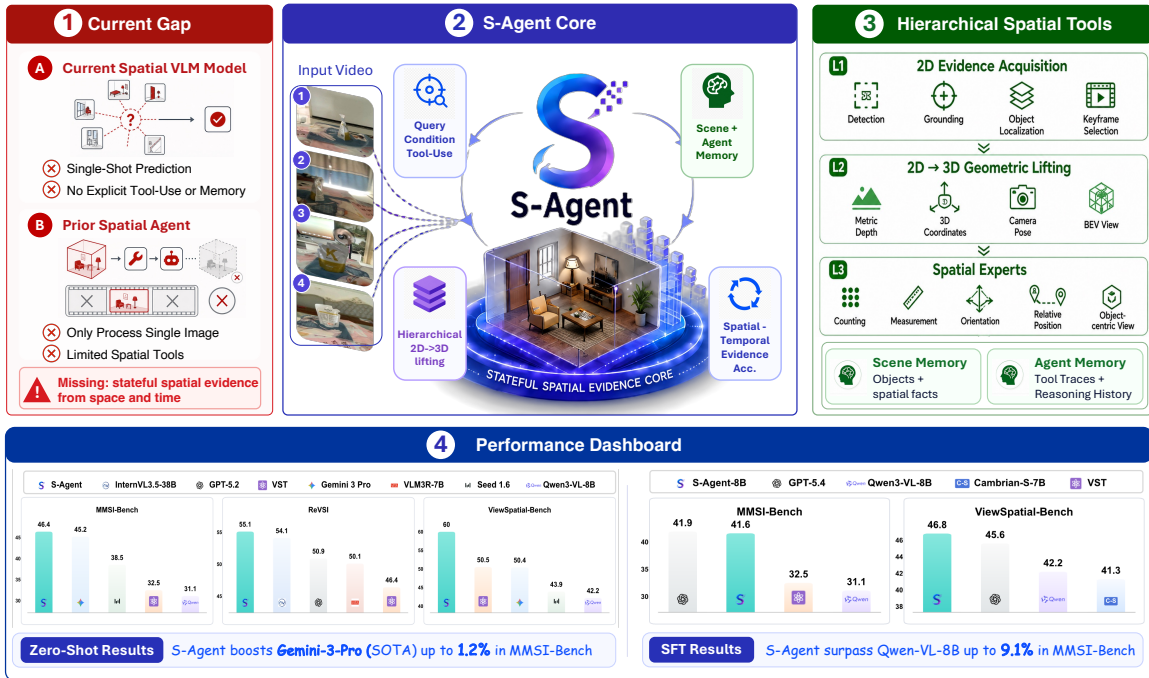


Figure 1: **Overview of S-Agent.** S-Agent is the first spatial tool-use agentic paradigm designed for continuous multi-view image and video reasoning, which formulates spatial reasoning as an active process of spatio-temporal evidence accumulation. It contains a VLM semantic planner with a hierarchy of spatial tools to ground, lift, and aggregate geometric cues, alongside a dual-memory system to maintain the evolving scene and reasoning history. Extensive experiments shows that our paradigm consistently enhances zero-shot VLMs and distills a compact agent (S-Agent-8B) that rivals advanced closed-source models.

## Abstract

Real-world spatial intelligence requires reasoning over continuous and evolving 3D world, yet existing VLMs and tool-augmented agents largely remain tied to static, stateless inference from isolated visual observations. We introduce **S-AGENT**, the first spatial tool-use agentic paradigm for continuous multi-view images and video understanding and reasoning. By formulating spatial reasoning as spatio-temporal evidence accumulation rather than isolated frame-level prediction, S-AGENT reshapes

spatial perception into scene-centric understanding beyond frame-centric recognition. Specifically, S-AGENT casts the VLM as a semantic planner that decides what evidence is needed, while a hierarchy of spatial tools and experts grounds objects in 2D, lifts them into 3D geometric evidence, and aggregates this evidence into high-level spatial knowledge (e.g., counting, measurement, orientation, and relative position). Complementarily, a temporal memory mechanism, including Scene Memory for maintaining the evolving scene state and Agent Memory for accumulating reasoning context, enables evidence integration across frames and reasoning steps. Comprehensive experiments on multi-view and video spatial reasoning benchmarks show that S-AGENT consistently improves both open-source and closed-source VLMs in a training-free manner. Beyond inference-time augmentation, supervised fine-tuning (SFT) on S-AGENT-generated spatial trajectories S-300K yield S-AGENT-8B, a compact spatial agent that significantly surpasses similar-scale baselines (e.g., Qwen3-VL-8B) and performs comparably to advanced closed-source models (e.g., GPT-5.4 and Gemini3-Pro).

## 1. Introduction

Spatial intelligence, the ability to understand geometric relationships among objects and their 3D environments, is essential for vision-language models (VLMs) to operate in the physical world and represents a key step toward artificial general intelligence (AGI), where models are expected to perceive, reason, and make decisions in 3D space as humans do. Such capability is crucial for real-world applications, including embodied robotics [8, 2], AR/VR perception [19], and autonomous driving [11, 4]. However, unlike humans naturally integrate visual cues into coherent 3D scene understanding, current VLMs are primarily trained on passive 2D visual-text corpora, with limited explicit 3D supervision or embodied experience [21, 1, 16]. This creates a fundamental semantic-to-geometric gap: while VLMs excel at probabilistic and qualitative semantic inference, their reasoning is often mediated by lossy semantic representations that fail to faithfully capture high-fidelity geometry, leaving them susceptible to textual patterns and semantic priors rather than grounded 3D spatial understanding in spatial reasoning [7, 3].

Recent advances in agentic VLMs substantially push the boundary of spatial understanding by augmenting VLMs with external tools, executable programs, and explicit geometric structure. For example, VADAR [18] dynamically constructs a Python API and synthesizes programs for 3D spatial reasoning, SpaceTools [6] trains VLMs to coordinate multiple vision and robotic tools through interactive reinforcement learning. However, despite their strong performance, these methods still largely focus on static images or isolated visual observations, which remains far from the goal of real-world spatial intelligence: *the real 3D world is hidden, evolving, and continuously projected into streams of 2D observations*. Reasoning from isolated 2D views alone makes it fundamentally challenging to maintain persistent object states, integrate evidence across viewpoints and time, and build a coherent understanding of the underlying 3D scene.

To move beyond static and stateless spatial reasoning, we introduce **S-AGENT**, a Spatial tool-use agentic paradigm for continuous multi-view images/video understanding. Our key motivation is that the missing ingredient for video-based spatial intelligence is not merely stronger 2D/3D visual recognition, but a reasoning mechanism that can accumulate spatial evidence along both spatial and temporal dimensions. Specifically, in continuous multi-view and video settings, each frame is only a partial and transient observation of the scene, while the key to spatial intelligence is to connect these observations into a spatially structured and

temporally persistent understanding of the underlying 3D world. Rather than asking VLMs to implicitly internalize this entire process, our S-AGENT casts the VLM as a semantic planner that decides what evidence is needed, while spatial tools/experts and temporal memory strategy provide continuous and explicit 3D awareness of the specific scene, ranging from low-level 2D/3D evidence (e.g., object grounding, depth information) to high-level spatial knowledge (e.g., orientations, relationship). This separation enables the agent to reason from accumulated evidence instead of isolated visual impressions, extending existing spatial agent methods toward evolving scene stateful, temporally grounded spatial scene understanding.

Motivated by this perspective, S-AGENT is designed as a VLM-orchestrated spatio-temporal reasoning framework: it progressively aggregates spatial evidence from fragmented 2D observations into structured 3D scene knowledge, while persistently accumulating temporal evidence across frames and reasoning iterations. (1) For the *spatial dimension*, S-AGENT follows a hierarchical understanding process. At the first level, **2D perception tools** ground objects and regions in individual frames, establishing object-centric visual facts for subsequent reasoning. At the second level, **multi-view 3D tools** enrich these grounded entities with geometric cues (e.g., depth, coordinates and camera pose), allowing evidence from different viewpoints to be integrated beyond the original image plane. At the third level, **specialized spatial experts** aggregate these geometric signals into higher-level spatial knowledge (e.g., object counts, physical measurements, orientations, and relative positions). (2) For the *temporal dimension*, S-AGENT maintains memory over the evolving reasoning process: **Scene Memory** tracks grounded entities across frames to preserve object identity and suppress duplicate evidence, while **Agent Memory** stores accumulated tool observations and intermediate reasoning traces for iterative refinement. In this way, S-AGENT turns video spatial reasoning from disconnected frame-level prediction into evidence accumulation over an evolving 3D scene.

Comprehensive experiments on the ViewSpatial-Bench [12] as well as video spatial reasoning benchmarks (MMSI-Video-Bench [15], ReVSI [38], and VSI-SUPER [31]) validate the robustness and generalizability of our approach. (1) **Zero-shot setting**. We directly instantiate S-AGENT with both open-source models (e.g., Qwen3) and closed-source APIs (e.g., Gemini and GPT) in a training-free manner. Simply and directly applying the S-AGENT framework consistently improves the spatial reasoning ability of these VLMs, with GPT-5.4 achieving up to a 10% gain on MMSI-Bench. (2) **Training setting**. Beyond inference-time improvement, we further construct a spatial-instruction dataset S-300K from zero-shot S-AGENT trajectories on the Sensenova-800K [5] training set (which is fully disjoint from all evaluation benchmarks) and use it to perform supervised fine-tuning on Qwen3-8B, resulting in S-AGENT-8B. Compared with direct Qwen3-8B inference, S-AGENT-8B achieves nearly a 15% improvement on MMSI-Bench, and performs comparably to most advanced closed-source models such as GPT-5.4 and Gemini3-Pro across multiple benchmarks. These results show that S-AGENT is not only an effective training-free inference framework, but also a scalable paradigm for building compact spatially capable agents.

## 2. Method

This section details the design of S-AGENT. We first formulate spatial reasoning as iterative updates to a scene state and an agent state in Section 2.1. We then describe how S-AGENT acquires hierarchical spatial evidence in Section 2.1.1, maintains temporal memory for stateful reasoning in Section 2.1.2, and describe how S-AGENT trajectories are used to train compact agents in Section 2.2.

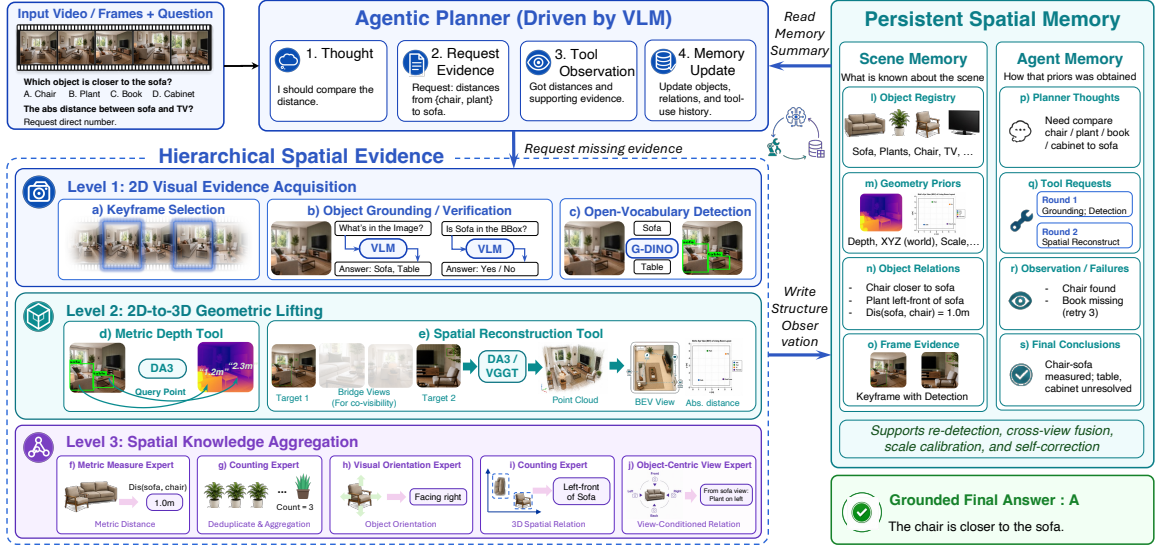


Figure 2 | The pipeline of S-AGENT. Instead of answering from an isolated visual impression, S-AGENT uses a VLM as a semantic planner, spatial tools and experts as scene-specific evidence providers, and memory as the carrier of persistent 3D state across views, frames, and reasoning steps.

## 2.1. S-Agent Framework

We consider spatial reasoning problems defined by a question  $q$  and a sequence or set of visual observations  $\mathcal{F}$ . The input can be a video (e.g., the scene and camera may evolve over time) or a multi-view image set (e.g., different images capture the same scene from different viewpoints). The goal of S-AGENT is to produce an answer  $a$  that depends on the underlying 3D scene state rather than on a single 2D projection. To this end, S-AGENT performs inference as an iterative evidence-seeking process, progressively acquiring and reusing scene-specific spatial evidence, as illustrated in Figure 2.

At reasoning step  $t$ , S-AGENT maintains two memory states. The first is a *scene memory state*  $\mathcal{S}_t$  for grounded spatial evidence, which stores grounded entities and their accumulated spatial attributes. The second is an *agent memory state*  $\mathcal{H}_t$  for reasoning history, which records previous tool calls, observations, and reasoning decisions. A tool-calling VLM planner  $\pi_\theta$  maps the question  $q$ , input observations  $\mathcal{F}$ , and current memory states ( $\mathcal{S}_t$  and  $\mathcal{H}_t$ ) to an evidence request  $r_t$ :

$$r_t = \pi_\theta(q, \mathcal{F}, \mathcal{S}_t, \mathcal{H}_t).$$

A spatial tool or expert executes  $r_t$  and returns an observation  $o_t$ , which is used to update both memory states:

$$(\mathcal{S}_{t+1}, \mathcal{H}_{t+1}) = \text{Update}(\mathcal{S}_t, \mathcal{H}_t, r_t, o_t).$$

The agent terminates when the accumulated evidence is sufficient to answer  $q$ . This formulation separates *semantic planning* from *spatial evidence acquisition*: the VLM decides what to measure or compare, while tools and memory provide scene-specific spatio-temporal evidence for the final reasoning. Unlike fixed pipelines or standard tool-use agents that treat each tool call as an isolated action, S-AGENT conditions each evidence request on both the question and the evolving memory state. As a result, perception and geometric computation are invoked on demand, and their outputs remain reusable across later reasoning steps. The following sections describe the two core mechanisms of this framework: hierarchical spatial evidence acquisition (Section 2.1.1) and temporal memory for stateful reasoning (Section 2.1.2).

### 2.1.1. Hierarchical Spatial Evidence

S-AGENT acquires spatial evidence through a three-level hierarchy that transforms raw 2D observations into explicit, scene-specific spatial knowledge. This hierarchy reflects the varying levels of evidence required by spatial tasks: some questions can be answered from localized image-level cues, while others require lifting those cues into 3D geometry or aggregating them through specialized spatial experts. This staged design keeps the VLM focused on semantic planning, while delegating scene-specific perception and spatial computation (e.g., visual localization, geometric recovery, and metric or relational computation) to tools whose outputs can also be stored and reused in memory.

We denote the three tool levels as  $\mathcal{T}^{(1)}$ ,  $\mathcal{T}^{(2)}$ , and  $\mathcal{T}^{(3)}$ , corresponding to 2D visual evidence acquisition, 2D-to-3D geometric lifting, and spatial knowledge aggregation, respectively. Given an evidence request  $r_t$ , S-AGENT selects a tool or expert  $T^{(k)} \in \mathcal{T}^{(k)}$  and produces an observation

$$o_t = T^{(k)}(r_t, \mathcal{F}, \mathcal{S}_t), \quad k \in \{1, 2, 3\}.$$

Depending on the selected level,  $o_t$  may contain localized image-level cues, lifted 3D geometry, or high-level spatial knowledge.

**Level 1: 2D Visual Evidence Acquisition** (Figure 2(a-c)). The first level identifies what visual evidence should be extracted from the raw 2D observations before higher-level spatial reasoning. Since videos or multi-view images contain many redundant, partial, or irrelevant views, S-AGENT first gathers query-relevant image-level cues, such as selecting informative frames, grounding referred entities with VLMs, and localizing candidate regions with open-vocabulary detectors. These image-level cues can directly support simple queries, while also serving as observations for subsequent 3D lifting and spatial reasoning.

**Level 2: 2D-to-3D Geometric Lifting** (Figure 2(d-e)). The second level lifts image-level evidence into a 3D-aware representation of the scene. Given the cues collected at Level 1, S-AGENT invokes multi-view geometric tools to recover static 3D information, such as depth structure, metric coordinates, camera poses, and bird’s-eye-view or novel-view evidence. This geometric lifting allows the agent to reason beyond the original image plane: fragmented 2D observations can be compared in a shared spatial context, apparent 2D size can be disambiguated from physical scale, and spatial relations can be evaluated with respect to camera motion or alternative viewpoints.

**Level 3: Spatial Knowledge Aggregation** (Figure 2(f-j)). The third level abstracts the 2D and 3D cues collected in the previous stages into high-level, scene-specific spatial knowledge. To this end, S-AGENT uses a set of specialized spatial experts, each responsible for a particular class of spatial queries, including counting, relative direction, object orientation, and physical size/distance. These experts aggregate the relevant evidence and return structured observations that can be directly consumed by the VLM planner for final reasoning. This design turns fragmented perceptual and geometric cues into explicit spatial knowledge of the current scene, reducing the need for the VLM to perform unreliable metric or relational reasoning in free-form text.

The implementation details of the tools and experts used in Levels 1-3 are provided in Appendix B.

### 2.1.2. Temporal Memory for Stateful Reasoning

To support stateful reasoning over continuous observations, S-AGENT maintains two complementary memories: *Scene Memory* for reusable scene evidence and *Agent Memory* for the

reasoning process. Each tool or expert observation from Section 2.1.1 updates both memories in different ways: its scene-relevant content is consolidated into Scene Memory, while the request, returned observation, and reasoning context are recorded in Agent Memory. This separation allows the VLM planner to reason over accumulated spatial knowledge while keeping track of what has been tried, what remains uncertain, and what evidence should be requested next.

Formally, after executing request  $r_t$  and receiving observation  $o_t$ , each tool observation is decomposed into reusable scene evidence  $e_t$  and process context  $c_t$ . The two memories are then updated with different operations:

$$\mathcal{S}_{t+1} = \text{Merge}(\mathcal{S}_t, e_t), \quad \mathcal{H}_{t+1} = \text{Append}(\mathcal{H}_t, c_t).$$

Scene Memory merges  $e_t$  into the current scene state, either by updating an existing entry or creating a new one, while Agent Memory appends  $c_t$  to the reasoning trajectory.

**Scene Memory** (Figure 2(l-o)). Scene Memory turns 2D/3D cues into a persistent, scene-level understanding. In multi-view images or videos, the same object may appear across different frames, viewpoints, scales, and referring expressions. Without a persistent memory, reasoning over these cues independently would lead to duplicated evidence and unstable object identity. Scene Memory therefore consolidates scene-relevant tool/expert observations into an evolving, entity-centric memory, binding repeated observations to persistent scene entities and accumulating their visual and geometric evidence over time. It is not a dense reconstruction of the full environment, but a question-conditioned spatial memory that preserves the evidence needed for the current query.

Concretely, Scene Memory stores two types of reusable content: grounded entities and derived spatial facts. For entities, the memory stores their textual aliases, supporting frames, localized visual evidence, and accumulated geometric attributes. For derived facts, it stores spatial relations or measurements computed by higher-level experts, together with the evidence from which they are derived. When a new observation arrives, S-AGENT either links it to an existing scene memory entry or creates a new one, allowing later reasoning steps to reuse previously grounded evidence or facts instead of re-processing each frame from scratch.

**Agent Memory** (Figure 2(p-s)). Agent Memory preserves the reasoning process that leads to the evolving scene understanding. In iterative tool-use reasoning, the agent should remember not only what has been observed, but also what has already been tried, which evidence was requested, which tools succeeded or failed, and why the planner decided to continue. Without such process memory, the planner may repeatedly issue redundant tool calls, overlook unresolved uncertainties, or contradict its earlier observations. Agent Memory therefore records the reasoning trajectory across iterations, providing the planner with a compact context for deciding the next evidence request.

Specifically, Agent Memory stores the planner’s intermediate thoughts, issued tool calls, returned observations, failure messages, and intermediate conclusions. Unlike Scene Memory, which consolidates reusable scene evidence, Agent Memory keeps the procedural context around how that evidence was obtained and used. When the planner receives a new memory summary, it can identify missing evidence, revisit uncertain observations, or refine its strategy based on previous tool feedback.

## 2.2. Training-Time Distillation

Beyond on inference-time reasoning, S-AGENT can also serve as a teacher for training compact spatial agents. We construct training data from SenseNova-SI-800K [5] by selecting samples that are both challenging for a weaker student model and likely to require tool use.



Table 1 | Detailed MMSI-Bench results. We follow the [32] taxonomy and group dimensions into Positional Relationship, Geometric Attribute, Motion Perception, and Multi-step Reasoning (MSR). C/O/R denote camera/object/region in positional-relation subcategories. *SenseNova* is abbreviated as *SN*. Top-1/2/3 results are highlighted in **deep**, **medium**, and **light** lavender.

| Model                      | Positional Relationship |             |            |             |             |             | Geometric Attr. |             | Motion Perception |             | MSR        | Avg.        |
|----------------------------|-------------------------|-------------|------------|-------------|-------------|-------------|-----------------|-------------|-------------------|-------------|------------|-------------|
|                            | C-C                     | O-O         | R-R        | C-O         | O-R         | C-R         | Meas.           | Appr.       | Cam.              | Obj.        |            |             |
| Proprietary Models         |                         |             |            |             |             |             |                 |             |                   |             |            |             |
| Gemini 3 Pro               | 47.3                    | 48.9        | 42.0       | 43.0        | 37.6        | 60.2        | 64.1            | 39.4        | 41.9              | 47.4        | 37.9       | 45.2        |
| Gemini 2.5 Pro             | 38.7                    | 34.0        | 40.7       | 44.2        | 38.8        | 41.0        | 62.5            | 30.3        | 39.2              | 25.0        | 33.3       | 38.0        |
| GPT-5.4                    | 41.9                    | 33.0        | 35.8       | 49.8        | 42.4        | 68.7        | 54.7            | 37.4        | 28.3              | 40.8        | 36.4       | 41.9        |
| Grok 4                     | 36.6                    | 35.1        | 39.5       | 34.9        | 45.9        | 50.6        | 21.9            | 22.7        | 40.5              | 43.4        | 38.4       | 37.8        |
| Open-weight General Models |                         |             |            |             |             |             |                 |             |                   |             |            |             |
| Seed 1.6                   | 36.6                    | 36.2        | 32.1       | 32.6        | 42.4        | 46.9        | 48.4            | 33.0        | 31.1              | 42.1        | 40.4       | 38.5        |
| InternVL3_5-8B             | 29.0                    | 26.6        | 29.6       | 24.4        | 31.8        | 25.3        | 29.7            | 25.8        | 14.9              | 34.2        | 36.4       | 29.0        |
| SN-U1-8B-MoT               | 46.2                    | 41.5        | 29.6       | 58.1        | 38.8        | 63.9        | 43.8            | 21.2        | 25.7              | 31.6        | 26.8       | 38.0        |
| Qwen3-VL-8B-Instruct       | 28.0                    | 37.2        | 32.1       | 31.4        | 35.3        | 38.5        | 37.5            | 15.2        | 27.0              | 28.9        | 29.8       | 31.1        |
| Qwen3-VL-8B-Thinking       | 31.2                    | 26.6        | 32.1       | 29.1        | 32.9        | 30.1        | 50.0            | 16.7        | 17.6              | 23.7        | 27.3       | 28.6        |
| Qwen3.5-9B                 | 34.4                    | 36.2        | 34.6       | 39.5        | 38.8        | 54.2        | 56.3            | 28.8        | 36.5              | 26.3        | 28.8       | 36.5        |
| Qwen3-VL-30B-A3B-Thinking  | 23.7                    | 31.9        | 35.8       | 31.4        | 36.5        | 22.9        | 40.6            | 19.7        | 18.9              | 27.6        | 31.3       | 29.4        |
| Open-weight Spatial Models |                         |             |            |             |             |             |                 |             |                   |             |            |             |
| SN-SI-1.1-Qwen2.5VL-7B     | 51.6                    | 29.8        | 32.1       | 50.0        | 29.4        | 42.2        | 37.5            | 28.8        | 23.0              | 34.2        | 18.7       | 32.8        |
| SN-SI-1.1-Qwen3VL-8B       | 44.1                    | 38.3        | 33.3       | 65.1        | 38.8        | 59.0        | 48.4            | 24.2        | 29.7              | 34.2        | 22.2       | 38.1        |
| VST-7B-SFT                 | 39.8                    | 36.2        | 35.8       | 37.2        | 29.4        | 33.7        | 29.7            | 47.0        | 36.5              | 35.5        | 18.2       | 32.5        |
| Ours (S-Agent)             | 46.2(+17.2)             | 43.6(+17.0) | 37.0(+7.4) | 43.0(+18.6) | 43.5(+11.7) | 63.9(+38.6) | 57.8(+28.1)     | 40.9(+15.1) | 46.0(+31.1)       | 48.7(+14.5) | 44.4(+8.0) | 46.4(+17.4) |

### 3.1. Experimental Setup

**Benchmarks.** We evaluate S-AGENT on four benchmarks that stress different forms of spatial reasoning across multi-image and video inputs. For *multi-image* reasoning, **MMSI-Bench** [32] provides multiple images of the same scene and tests whether models can integrate evidence across views for positional relationships, geometric attributes, motion perception, and multi-step spatial reasoning. **ViewSpatial-Bench** [12] focuses more specifically on perspective-aware localization, requiring models to localize objects or infer positions under different camera viewpoints. For *video reasoning*, **ReVSI** [38] evaluates 3D spatial reasoning from dynamic observations, emphasizing whether models can infer spatial relations that are not reliably recoverable from isolated frames. **VSI-SUPER** [31] focuses on video spatial change reasoning, requiring models to identify how objects, viewpoints, or spatial layouts change over time.

**Baselines.** We compare S-AGENT with three categories of baselines: advanced proprietary VLMs (e.g., *Gemini-3-Pro*, *GPT-5.4*, and *Grok 4*), open-weight general VLMs (e.g., *Qwen series*), and spatially specialized models (e.g., *Cambrian-S*, *VST-SFT*, and *SenseNova-SI series*). The first two groups measure performance against strong general-purpose multimodal systems, while the third evaluates whether S-AGENT can compete with models explicitly trained or tuned for spatial reasoning.

**Models.** In the zero-shot setting, we instantiate S-AGENT with advanced VLMs (GPT-5.4 and Gemini 3 Pro) as tool-calling planners, without any task-specific training. In the trained-agent setting, we use Qwen3-8B-Instruct as the backbone planner and train it on trajectories generated by zero-shot S-AGENT, yielding our compact agent S-AGENT-8B.

**Training Data.** We construct training data from SenseNova-800K [5], which is fully disjoint from all evaluation benchmarks. We randomly sample 100K questions and use zero-shot S-AGENT with a GPT-5.4 planner to generate tool-use trajectories. We then filter trajectories by execution validity and final-answer correctness, and decompose the retained trajectories into full final-answer samples, turn-level VLM-call samples, and expert/tool-specific samples. This yields 292,391 SFT samples, denoted as S-300K. Appendix C provides the detailed filtering criteria and data distribution.

Table 2 | Results on ViewSpatial-Bench [12]. We report the official five question types: camera-perspective object view orientation (C-OVO), camera-perspective relative direction (C-RD), person-perspective object view orientation (P-OVO), person-perspective relative direction (P-RD), and person-perspective scene-simulation relative direction (P-SSRD).

| Model                      | Camera Perspective |            | Person Perspective |             |             | Avg.        |
|----------------------------|--------------------|------------|--------------------|-------------|-------------|-------------|
|                            | C-OVO              | C-RD       | P-OVO              | P-RD        | P-SSRD      |             |
| Proprietary Models         |                    |            |                    |             |             |             |
| Gemini 3 Pro               | 31.6               | 61.9       | 41.1               | 74.4        | 38.9        | 50.4        |
| Gemini-2.5-Pro             | 33.0               | 59.1       | 51.0               | 45.8        | 32.6        | 46.1        |
| GPT-5.4                    | 27.9               | 60.2       | 41.0               | 48.5        | 40.1        | 45.6        |
| Grok-4                     | 23.9               | 57.1       | 47.6               | 51.7        | 24.9        | 43.2        |
| Open-weight General Models |                    |            |                    |             |             |             |
| Seed 1.6                   | 26.9               | 55.8       | 54.8               | 48.5        | 26.6        | 43.9        |
| Qwen3-VL-8B-Instruct       | 29.7               | 54.2       | 47.3               | 40.3        | 31.1        | 42.2        |
| BAGEL-7B-MoT               | 38.7               | 48.3       | 47.0               | 42.5        | 26.5        | 41.3        |
| InternVL3_5-8B             | 24.7               | 49.8       | 50.3               | 34.6        | 32.9        | 40.0        |
| Open-weight Spatial Models |                    |            |                    |             |             |             |
| Cambrian-S-7B              | 22.7               | 50.4       | 45.0               | 38.8        | 41.9        | 41.3        |
| VST-3B-SFT                 | 35.4               | 46.9       | 70.3               | 52.6        | 62.8        | 52.9        |
| VST-7B-SFT                 | 29.6               | 52.7       | 51.9               | 50.7        | 64.5        | 50.5        |
| SN-SI-1.1-Qwen2.5VL-7B     | 26.7               | 47.9       | 57.1               | 43.2        | 49.7        | 45.5        |
| SN-SI-1.1-Qwen3VL-8B       | 22.0               | 60.3       | 67.8               | 41.5        | 55.6        | 51.2        |
| Ours (S-AGENT)             | 55.5(+27.6)        | 62.5(+2.3) | 42.2(+1.2)         | 81.1(+32.6) | 60.6(+20.5) | 60.0(+14.4) |

**Training Configuration.** We fine-tune Qwen3-VL-8B-Instruct on S-300K using LLaMA-Factory [40] with the qwen3\_vl\_nothink template on 8\*B200 GPUs. The model is trained with the standard supervised next-token prediction objective over assistant responses, including serialized tool-use trajectories, tool observations, and final answers. We use a maximum sequence length of 8192, a learning rate of  $5 \times 10^{-5}$ , cosine learning-rate decay with 3% warmup, and train for one epoch. The resulting compact spatial agent is denoted as S-AGENT-8B.

### 3.2. Zero-Shot Performance.

We report the results on MMSI-Bench and ViewSpatial-Bench in the main text, while the results on ReVSI and VSI-SUPER are provided in the Appendix D.

**Results on MMSI-Bench.** Table 1 shows that our S-AGENT achieves the best overall zero-shot performance on MMSI-Bench, obtaining the highest average score of 46.4%. It outperforms the strongest proprietary baseline Gemini-3-Pro by 1.2%, and surpasses GPT-5.4 by 4.5%. Notably, S-AGENT achieves the best results on both motion perception subtasks, i.e., camera motion (46.0%) and object motion (48.7%), as well as multi-step reasoning (44.4%), while remaining competitive across positional and geometric categories. These results demonstrate the effectiveness of S-AGENT for zero-shot spatial reasoning, with particularly strong performance on dynamic motion understanding and multi-step reasoning while maintaining robust results across static spatial and geometric tasks.

**Results on ViewSpatial-Bench.** Table 2 reports the zero-shot results on ViewSpatial-Bench. S-AGENT achieves an average score of 60.0%, outperforming GPT-5.4 by 14.4%. It obtains the best performance on C-OVO (55.5%) and P-RD (81.1%), showing strong capability in both camera-centered and person-centered spatial reasoning. S-AGENT also brings large gains on the more challenging P-SSRD split, improving over GPT-5.4 by 20.5%. These results further demonstrate the effectiveness of S-AGENT for zero-shot view-aware spatial reasoning, especially when reasoning over relative directions and perspective-dependent spatial relations.

Table 3 | Trajectory distillation results across four spatial reasoning benchmarks. All scores are accuracy (%).

| Model                     | MMSI        | ViewSpatial | ReVSI       |
|---------------------------|-------------|-------------|-------------|
| <i>Proprietary VLMs</i>   |             |             |             |
| Gemini-3-Pro              | 45.2        | 50.4        | 60.9        |
| GPT-5.4                   | 41.9        | 45.6        | -           |
| <i>Open-weight Models</i> |             |             |             |
| Qwen3-VL-8B-Instruct      | 31.1        | 42.2        | 49.1        |
| S-AGENT (Qwen3-VL-8B)     | 30.7        | 44.1        | 49.5        |
| <b>S-AGENT-8B</b>         | <b>41.6</b> | <b>46.8</b> | <b>52.8</b> |

Table 4 | Ablation on ViewSpatial with S-AGENT using GPT-5.4 as the planner.

| S-AGENT                          | Evidence |    |    | Memory |       | Avg.        |
|----------------------------------|----------|----|----|--------|-------|-------------|
|                                  | L1       | L2 | L3 | Scene  | Agent |             |
| <i>Spatial evidence ablation</i> |          |    |    |        |       |             |
| VLM-only                         |          |    |    |        |       | 45.6        |
| + Level-1 2D evidence            | ✓        |    |    |        |       | 49.0        |
| + Level-2 3D evidence            | ✓        | ✓  |    |        |       | 49.8        |
| + Level-3 3D experts             | ✓        | ✓  | ✓  |        |       | 56.7        |
| <i>Memory ablation</i>           |          |    |    |        |       |             |
| Spatial only                     | ✓        | ✓  | ✓  |        |       | 56.7        |
| + Scene memory                   | ✓        | ✓  | ✓  | ✓      |       | 58.2        |
| + Agent memory                   | ✓        | ✓  | ✓  |        | ✓     | 57.6        |
| Full S-AGENT                     | ✓        | ✓  | ✓  | ✓      | ✓     | <b>60.0</b> |

### 3.3. Trajectory Distillation from S-Agent

We evaluate whether the reasoning trajectories generated by S-AGENT can be used to train a smaller open-weight spatial agent. Specifically, we fine-tune Qwen3-VL-8B-Instruct on S-300K and obtain S-AGENT-8B. Table 3 compares S-AGENT-8B with proprietary VLMs, the original Qwen3-VL-8B-Instruct, and S-AGENT using the same Qwen3-VL-8B backbone. A key observation is that simply equipping the base Qwen3-VL-8B-Instruct with the S-AGENT framework does not consistently improve performance. The base 8B planner often struggles with tool selection and noisy tool observations, so tool use can bring limited gains or even hurt performance.

In contrast, S-AGENT-8B consistently improves over both the base Qwen3-VL-8B-Instruct and S-AGENT with the same 8B planner across the four benchmarks. This shows that trajectory distillation teaches not only spatial answers, but also reusable tool-use and evidence-integration patterns for spatial reasoning. Notably, S-AGENT-8B also achieves competitive performance compared with state-of-the-art proprietary models such as GPT-5.4 and Gemini 3 Pro.

### 3.4. Ablation Studies

We ablate the spatial evidence hierarchy and memory modules of S-AGENT on ViewSpatial using GPT-5.4 as the planner. As shown in Table 4, adding Level-1 2D evidence improves the VLM-only baseline from 45.6% to 49.2%, showing that explicit visual grounding provides useful support for spatial reasoning. However, directly adding Level-2 3D evidence does not further improve performance. We observe that raw 3D evidence often contains dense numerical information, such as camera poses, depth values, and noisy reconstructed points, which can be difficult for the VLM planner to interpret and may even distract it from the task-relevant spatial cues.

In contrast, enabling Level-3 3D experts substantially improves the score to 56.7%. This suggests that 3D evidence becomes most useful when it is filtered and interpreted by specialized experts, which convert noisy geometric outputs into task-oriented measurements, relative positions, or spatial conclusions. The memory modules provide additional gains: scene memory improves the score to 58.2%, agent memory improves it to 57.6%, and combining both yields the full S-AGENT score of 60.0%. These results show that S-AGENT benefits from both structured spatial evidence and persistent memory, with expert-mediated interpretation being crucial for effectively using 3D information.

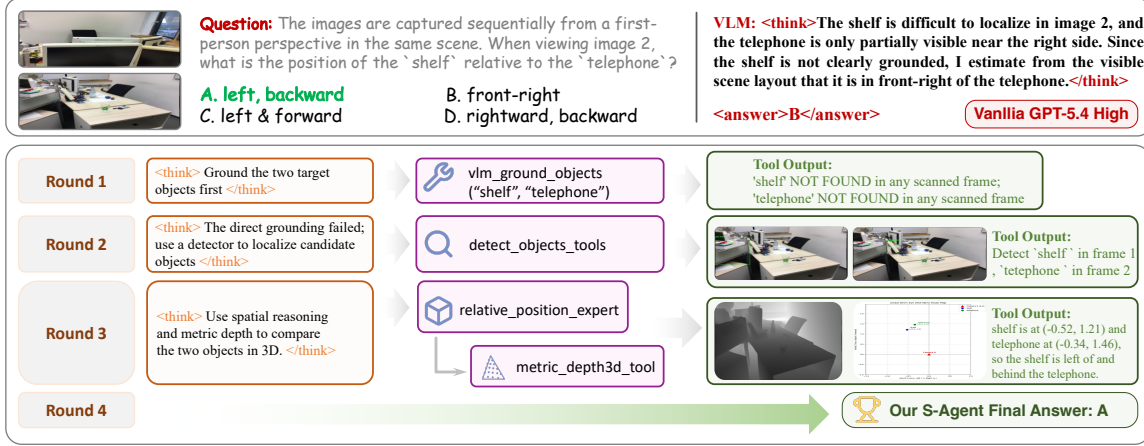


Figure 4 | Qualitative example of tool-grounded spatial reasoning. Unlike vanilla VLMs that fail on incomplete cues, our approach accurately infers 3D relations by our hierarchical spatial tools and a depth-guided position expert.

### 3.5. Qualitative Analysis

We provide qualitative examples to illustrate how S-AGENT obtains explicit spatial evidence before answering. Figure 4 shows a relative-position question in a first-person video. A direct VLM response struggles with this case because the queried objects are partially occluded, and not both clearly visible in the target view. Without grounded evidence, it relies on the apparent 2D layout and incorrectly guesses that the shelf is in the front-right direction.

In contrast, S-AGENT follows a tool-grounded trajectory. Although the initial grounding tool fails to locate both queried objects, the agent does not answer immediately. It instead issues targeted detection calls over the video frames, using both the original object names, “shelf” and “telephone”, and a semantically related query, “desk phone”. These calls recover usable boxes for the shelf and telephone. The relative-position expert then lifts the selected boxes into a metric 3D representation via the depth tool and constructs a bird’s-eye-view layout. In this layout, the shelf is estimated at  $(-0.52, 1.21)$  and the telephone at  $(-0.34, 1.46)$ . The recovered geometry shows that the shelf is to the left of and behind the telephone, leading to the correct answer.

## 4. Conclusion

We introduce S-AGENT, a spatial tool-use agentic framework for continuous multi-view image and video spatial reasoning. Instead of treating spatial reasoning as a single-shot prediction from isolated visual inputs, S-AGENT formulates it as a process of spatio-temporal evidence accumulation. It uses a VLM planner to actively acquire hierarchical spatial evidence, from 2D grounding to 3D geometric lifting and expert-level spatial knowledge, while maintaining scene and agent memories for stateful reasoning across views, frames, and tool-use steps. Extensive experiments show that S-AGENT consistently improves strong VLMs in a training-free zero-shot setting, especially on motion, perspective-aware, and multi-step spatial reasoning tasks. Furthermore, trajectories generated by S-AGENT can be distilled into S-AGENT-8B, enabling an open-weight 8B model to learn more reliable tool-use and spatial evidence integration. These results suggest that agentic evidence accumulation is a promising direction for building VLMs with stronger and more grounded spatial intelligence.

## References

- [1] Jean-Baptiste Alayrac, Jeff Donahue, Pauline Luc, Antoine Miech, Iain Barr, Yana Hasson, Karel Lenc, Arthur Mensch, Katie Millican, Malcolm Reynolds, Roman Ring, Eliza Rutherford, Serkan Cabi, Tengda Han, Zhitao Gong, Sina Samangooei, Marianne Monteiro, Jacob Menick, Sebastian Borgeaud, and 8 others. 2022. [Flamingo: a visual language model for few-shot learning](#). *Preprint*, arXiv:2204.14198.
- [2] Anthony Brohan, Noah Brown, Justice Carbajal, Yevgen Chebotar, Xi Chen, Krzysztof Choromanski, Tianli Ding, Danny Driess, Avinava Dubey, Chelsea Finn, Pete Florence, Chuyuan Fu, Montse Gonzalez Arenas, Keerthana Gopalakrishnan, Kehang Han, Karol Hausman, Alexander Herzog, Jasmine Hsu, Brian Ichter, and 35 others. 2023. [Rt-2: Vision-language-action models transfer web knowledge to robotic control](#). *Preprint*, arXiv:2307.15818.
- [3] Ellis Brown, Jihan Yang, Shusheng Yang, Rob Fergus, and Saining Xie. 2025. [Benchmark designers should "train on the test set" to expose exploitable non-visual shortcuts](#). *Preprint*, arXiv:2511.04655.
- [4] Holger Caesar, Varun Bankiti, Alex H. Lang, Sourabh Vora, Venice Erin Liong, Qiang Xu, Anush Krishnan, Yu Pan, Giancarlo Baldan, and Oscar Beijbom. 2020. nuscenes: A multimodal dataset for autonomous driving. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 11621–11631.
- [5] Zhongang Cai, Ruisi Wang, Chenyang Gu, Fanyi Pu, Junxiang Xu, Yubo Wang, Wanqi Yin, Zhitao Yang, Chen Wei, Qingping Sun, Tongxi Zhou, Jiaqi Li, Hui En Pang, Oscar Qian, Yukun Wei, Zhiqian Lin, Xuanke Shi, Kewang Deng, Xiaoyang Han, and 10 others. 2026. [Scaling spatial intelligence with multimodal foundation models](#). *Preprint*, arXiv:2511.13719.
- [6] Siyi Chen, Mikaela Angelina Uy, Chan Hee Song, Faisal Ladhak, Adithyavairavan Murali, Qing Qu, Stan Birchfield, Valts Blukis, and Jonathan Tremblay. 2025. [Spacetools: Tool-augmented spatial reasoning via double interactive rl](#). *Preprint*, arXiv:2512.04069.
- [7] Zeren Chen, Xiaoya Lu, Zhijie Zheng, Pengrui Li, Lehan He, Yijin Zhou, Jing Shao, Bohan Zhuang, and Lu Sheng. 2025. [Geometrically-constrained agent for spatial reasoning](#). *Preprint*, arXiv:2511.22659.
- [8] Danny Driess, Fei Xia, Mehdi S. M. Sajjadi, Corey Lynch, Aakanksha Chowdhery, Brian Ichter, Ayzaan Wahid, Jonathan Tompson, Quan Vuong, Tianhe Yu, Wenlong Huang, Yevgen Chebotar, Pierre Sermanet, Daniel Duckworth, Sergey Levine, Vincent Vanhoucke, Karol Hausman, Marc Toussaint, Klaus Greff, and 3 others. 2023. [Palm-e: An embodied multimodal language model](#). *Preprint*, arXiv:2303.03378.
- [9] Mengfei Du, Binhao Wu, Zejun Li, Xuanjing Huang, and Zhongyu Wei. 2024. [Embspatial-bench: Benchmarking spatial understanding for embodied tasks with large vision-language models](#). *Preprint*, arXiv:2406.05756.
- [10] Xingyu Fu, Yushi Hu, Bangzheng Li, Yu Feng, Haoyu Wang, Xudong Lin, Dan Roth, Noah A. Smith, Wei-Chiu Ma, and Ranjay Krishna. 2024. [Blink: Multimodal large language models can see but not perceive](#). *Preprint*, arXiv:2404.12390.
- [11] Andreas Geiger, Philip Lenz, and Raquel Urtasun. 2012. [Are we ready for autonomous driving? the kitti vision benchmark suite](#). In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 3354–3361.
- [12] Dingming Li, Hongxing Li, Zixuan Wang, Yuchen Yan, Hang Zhang, Siqi Chen, Guiyang Hou, Shengpei Jiang, Wenqi Zhang, Yongliang Shen, Weiming Lu, and Yueting Zhuang. 2025. [Viewspatial-bench: Evaluating multi-perspective spatial localization in vision-language models](#). *Preprint*, arXiv:2505.21500.
- [13] Hongxing Li, Dingming Li, Zixuan Wang, Yuchen Yan, Hang Wu, Wenqi Zhang, Yongliang Shen, Weiming Lu, Jun Xiao, and Yueting Zhuang. 2025. [Spatialladder: Progressive training for spatial reasoning in vision-language models](#). *Preprint*, arXiv:2510.08531.
- [14] Haotong Lin, Sili Chen, Junhao Liew, Donny Y. Chen, Zhenyu Li, Guang Shi, Jiashi Feng, and Bingyi Kang. 2025. [Depth anything 3: Recovering the visual space from any views](#). *Preprint*, arXiv:2511.10647.
- [15] Jingli Lin, Runsen Xu, Shaohao Zhu, Sihan Yang, Peizhou Cao, Yunlong Ran, Miao Hu, Chenming Zhu, Yiman Xie, Yilin Long, Wenbo Hu, Dahua Lin, Tai Wang, and Jiangmiao Pang. 2025. [Mmsi-video-bench: A holistic benchmark for video-based spatial intelligence](#). *Preprint*, arXiv:2512.10863.
- [16] Haotian Liu, Chunyuan Li, Qingyang Wu, and Yong Jae Lee. 2023. [Visual instruction tuning](#). *Preprint*, arXiv:2304.08485.
- [17] Wufei Ma, Haoyu Chen, Guofeng Zhang, Yu-Cheng Chou, Jieneng Chen, Celso M. de Melo, and Alan Yuille. 2025. [3dsrbench: A comprehensive 3d spatial reasoning benchmark](#). *Preprint*, arXiv:2412.07825.
- [18] Damiano Marsili, Rohun Agrawal, Yisong Yue, and Georgia Gkioxari. 2025. [Visual agentic ai for spatial reasoning with a dynamic api](#). *Preprint*, arXiv:2502.06787.

- [19] Richard A. Newcombe, Shahram Izadi, Otmar Hilliges, David Molyneaux, David Kim, Andrew J. Davison, Pushmeet Kohli, Jamie Shotton, Steve Hodges, and Andrew Fitzgibbon. 2011. [Kinectfusion: Real-time dense surface mapping and tracking](#). In *2011 10th IEEE International Symposium on Mixed and Augmented Reality*, pages 127–136. IEEE.
- [20] Kun Ouyang, Yuanxin Liu, Haoning Wu, Yi Liu, Hao Zhou, Jie Zhou, Fandong Meng, and Xu Sun. 2025. [Spacer: Reinforcing mllms in video spatial reasoning](#). *Preprint*, arXiv:2504.01805.
- [21] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, Gretchen Krueger, and Ilya Sutskever. 2021. [Learning transferable visual models from natural language supervision](#). *Preprint*, arXiv:2103.00020.
- [22] Dídac Surís, Sachit Menon, and Carl Vondrick. 2023. [Vipergpt: Visual inference via python execution for reasoning](#). *Preprint*, arXiv:2303.08128.
- [23] Vishaal Udandara, Shyamgopal Karthik, Surabhi S Nath, Andreas Hochlehnert, Matthias Bethge, and Ameya Prabhu. 2025. Solving spatial supersensing without spatial supersensing. *arXiv preprint arXiv:2511.16655*.
- [24] Jianyuan Wang, Minghao Chen, Nikita Karaev, Andrea Vedaldi, Christian Rupprecht, and David Novotny. 2025. [Vgggt: Visual geometry grounded transformer](#). *Preprint*, arXiv:2503.11651.
- [25] Qineng Wang, Baiqiao Yin, Pingyue Zhang, Jianshu Zhang, Kangrui Wang, Zihan Wang, Jieyu Zhang, Keshigeyan Chandrasegaran, Han Liu, Ranjay Krishna, Saining Xie, Jiajun Wu, Li Fei-Fei, and Manling Li. 2026. [Mindcube: Spatial mental modeling from limited views](#). *Preprint*, arXiv:2506.21458.
- [26] Chenfei Wu, Shengming Yin, Weizhen Qi, Xiaodong Wang, Zecheng Tang, and Nan Duan. 2023. [Visual chatgpt: Talking, drawing and editing with visual foundation models](#). *Preprint*, arXiv:2303.04671.
- [27] Diankun Wu, Fangfu Liu, Yi-Hsin Hung, and Yueqi Duan. 2025. [Spatial-mllm: Boosting mllm capabilities in visual-based spatial intelligence](#). *Preprint*, arXiv:2505.23747.
- [28] Junfei Wu, Jian Guan, Kaituo Feng, Qiang Liu, Shu Wu, Liang Wang, Wei Wu, and Tieniu Tan. 2025. [Reinforcing spatial reasoning in vision-language models with interwoven thinking and visual drawing](#). *Preprint*, arXiv:2506.09965.
- [29] Jihan Yang, Shusheng Yang, Anjali W. Gupta, Rilyn Han, Li Fei-Fei, and Saining Xie. 2025. [Thinking in space: How multimodal large language models see, remember, and recall spaces](#). *Preprint*, arXiv:2412.14171.
- [30] Rui Yang, Ziyu Zhu, Yanwei Li, Jingjia Huang, Shen Yan, Siyuan Zhou, Zhe Liu, Xiangtai Li, Shuangye Li, Wenqian Wang, Yi Lin, and Hengshuang Zhao. 2025. [Visual spatial tuning](#). *Preprint*, arXiv:2511.05491.
- [31] Shusheng Yang, Jihan Yang, Pinzhi Huang, Ellis Brown, Zihao Yang, Yue Yu, Shengbang Tong, Zihan Zheng, Yifan Xu, Muhan Wang, Daohan Lu, Rob Fergus, Yann LeCun, Li Fei-Fei, and Saining Xie. 2025. [Cambrian-s: Towards spatial supersensing in video](#). *Preprint*, arXiv:2511.04670.
- [32] Sihan Yang, Runsen Xu, Yiman Xie, Sizhe Yang, Mo Li, Jingli Lin, Chenming Zhu, Xiaochen Chen, Haodong Duan, Xiangyu Yue, Dahua Lin, Tai Wang, and Jiangmiao Pang. 2025. [Mmsi-bench: A benchmark for multi-image spatial intelligence](#). *Preprint*, arXiv:2505.23764.
- [33] Zhengyuan Yang, Linjie Li, Jianfeng Wang, Kevin Lin, Ehsan Azarnasab, Faisal Ahmed, Zicheng Liu, Ce Liu, Michael Zeng, and Lijuan Wang. 2023. [Mm-react: Prompting chatgpt for multimodal reasoning and action](#). *Preprint*, arXiv:2303.11381.
- [34] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. [React: Synergizing reasoning and acting in language models](#). *Preprint*, arXiv:2210.03629.
- [35] Jinhui Ye, Zihan Wang, Haosen Sun, Keshigeyan Chandrasegaran, Zane Durante, Cristobal Eyzaguirre, Yonatan Bisk, Juan Carlos Niebles, Ehsan Adeli, Li Fei-Fei, Jiajun Wu, and Manling Li. 2025. [T\\*: Re-thinking temporal search for long-form video understanding](#). *Preprint*, arXiv:2504.02259.
- [36] Hang Zhang, Xin Li, and Lidong Bing. 2023. [Video-llama: An instruction-tuned audio-visual language model for video understanding](#). *Preprint*, arXiv:2306.02858.
- [37] Peiyuan Zhang, Kaichen Zhang, Bo Li, Guangtao Zeng, Jingkang Yang, Yuanhan Zhang, Ziyue Wang, Haoran Tan, Chunyuan Li, and Ziwei Liu. 2024. [Long context transfer from language to vision](#). *Preprint*, arXiv:2406.16852.
- [38] Yiming Zhang, Jiacheng Chen, Jiaqi Tan, Yongsun Mao, Wenhui Chen, and Angel X. Chang. 2026. [Revs: Rebuilding visual spatial intelligence evaluation for accurate assessment of vlm 3d reasoning](#). *Preprint*, arXiv:2604.24300.

- [39] Zaibin Zhang, Yuhan Wu, Lianjie Jia, Yifan Wang, Zhongbo Zhang, Yijiang Li, Binghao Ran, Fuxi Zhang, Zhuohan Sun, Zhenfei Yin, and 1 others. 2026. Think3d: Thinking with space for spatial reasoning. *arXiv preprint arXiv:2601.13029*.
- [40] Yaowei Zheng, Richong Zhang, Junhao Zhang, Yanhan Ye, Zheyang Luo, Zhangchi Feng, and Yongqiang Ma. 2024. **Llamafactory: Unified efficient fine-tuning of 100+ language models**. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 3: System Demonstrations)*, Bangkok, Thailand. Association for Computational Linguistics.

## Appendix

### A. Related Work

**Spatial Intelligence in VLMs.** Recent work has sought to improve the spatial intelligence of VLMs by scaling spatial supervision, introducing geometry-aware architectures, or designing spatially focused training objectives. Cambrian-S [31] and SenseNova-SI [5] construct large-scale spatial instruction data, while Spatial-MLLM [27] and VST [30] inject explicit spatial modeling or visual spatial tuning into multimodal backbones. Other works, such as SpaceR [20], ViLaSR [28], MindCube [25], and SpatialLadder [13], further improve spatial reasoning through reinforcement learning, verifiable rewards, or curriculum design. These efforts have advanced performance on spatial benchmarks such as BLINK [10], 3DSR [17], EmbSpatial [9], MMSI-Bench [32], and VSI-Bench [29]. However, most of them remain *training-driven and single-shot*: the model is expected to encode spatial capability into its parameters and produce an answer in one forward pass, relying on the model’s internalized spatial knowledge rather than explicit, scene-specific evidence acquisition at inference time.

**Agentic Spatial Reasoning.** Tool-use agents extend language and vision-language models by interleaving reasoning with calls to external tools, as shown in general agent frameworks such as ReAct [34] and visual tool-use systems such as ViperGPT [22], Visual ChatGPT [26], and MM-ReAct [33]. More recent work brings this paradigm to spatial reasoning by equipping VLMs with explicit geometric tools or structured computation. VADAR [18] synthesizes Python programs over dynamically constructed 3D APIs, SpaceTools [6] trains VLMs to coordinate vision and robotic tools through reinforcement learning, and GCA [7] constrains the reasoning process with formal reference-frame and objective constraints before deterministic geometric computation. Concurrent to these efforts, Think3D [39] equips VLM agents with 3D reconstruction and camera-manipulation tools, enabling active exploration through ego/global-view switching and novel-view rendering. These methods demonstrate the promise of agentic spatial reasoning, but they are still limited in capturing the continuous nature of human spatial understanding, where partial observations are integrated over time, object states are maintained across viewpoints, and spatial judgments are made from an evolving scene representation.

**Long-video and Multi-view Understanding** methods commonly handle continuous observations through frame compression or reconstruction-first pipelines. Frame-compression methods sample, retrieve, or summarize a limited set of frames before feeding them to long-context VLMs [36, 37, 35], improving efficiency but risking the loss of question-relevant spatial evidence. Reconstruction-first methods instead build an explicit 3D representation using multi-view geometry or feed-forward reconstruction models [24, 14], providing stronger geometric grounding but often incurring unnecessary computation when the query only requires sparse or localized evidence. However, the selected frames or reconstructed geometry are typically consumed as fixed context, leaving spatial grounding, cross-view association, and metric comparison largely to implicit reasoning or a separate downstream step. Thus, they improve access to visual or geometric information, but do not fully close the loop between evidence acquisition, spatial computation, and persistent scene-level reasoning.

### B. Details of Tools and Experts

**Level 1 tools.** Level 1 contains tools for extracting query-relevant evidence from raw 2D observations. The `detect_objects_tool` performs open-vocabulary 2D object detection using GroundingDINO. Given an image path and a text prompt, it returns bounding boxes,

confidence scores, predicted labels, textual location descriptions, and a visualization with detected boxes. This tool converts entities mentioned in the question into localized 2D regions, which serve as the basis for later measurement, counting, and relative-position reasoning.

The `vlm_ground_objects` tool performs multi-frame or multi-image grounding for target entities. It uses a two-stage procedure: first, a VLM performs a visibility vote over candidate frames to determine where the target is visible; second, `detect_objects_tool` is applied to the selected best frame to obtain the final bounding box. The tool returns the best supporting frame, bounding box, VLM confidence, detector confidence, and visualization for each target. This is useful when the target may appear across multiple views or when the referring expression is too complex for direct single-frame detection.

The `depth_estimation_tool` provides lightweight image-level depth cues. Given a single image and optional query points, it returns a depth-map visualization and depth estimates at the specified locations. We use this tool to support simple depth, occlusion, and front/back reasoning at the image level. It should be distinguished from Level 2 geometric lifting, as it provides local image-level depth evidence rather than a full 3D scene representation.

For videos, S-AGENT further uses frame or keyframe selection tools, such as `TStarKeyframeSearchTool`, to identify informative frames before applying the above image-level tools. This reduces redundant visual input and allows the agent to focus subsequent grounding and perception on frames that are most relevant to the current question.

**Level 2 tools.** Level 2 contains tools for lifting localized 2D evidence into metric 3D geometry. The main tool in our current implementation is `metric_depth3d_tool`, which is built on Depth-Anything-3. Given multiple images and query points or boxes, it estimates metric depth, 3D coordinates, camera poses, and depth visualizations. This tool provides the shared 3D geometric substrate used by downstream spatial experts, especially the Metric Measurement Expert and Relative Position Expert. In the current no-VGGT implementation, `metric_depth3d_tool` is the core Level-2 module for stable 2D-to-3D lifting.

**Level 3 experts.** Level 3 consists of five specialized spatial experts: the Metric Measurement Expert, Counting Expert, Visual Orientation Expert, Relative Position Expert, and Object-Centric View Expert. Each expert integrates the 2D evidence from Level 1 and, when needed, the lifted 3D evidence from Level 2 to produce structured, scene-specific spatial knowledge for the planner.

- **Metric measurement expert** serves as a geometry-grounded measurement specialist that estimates explicit spatial quantities (e.g., camera-to-object distance, object-to-object distance, and physical object size). Given target entities specified by the planner, it first reuses or obtains Level-1 evidence as normalized object boxes, and then queries the Level-2 geometric module to recover metric 3D points inside these regions. The expert deterministically maps the request to a measurement route, such as *closest-point distance*, *center-to-center distance*, or *longest object dimension*, samples representative points from the grounded boxes, and computes the final value from their recovered 3D coordinates. It returns a structured observation containing the measurement type, numerical value, unit, confidence, and supporting regions.
- **Counting expert** serves as a detection-grounded aggregation specialist that answers object-counting queries, including single-object counts and condition-aware counts over multiple frames. Given target entities or counting constraints specified by the planner, it first reuses

or obtains Level-1 evidence by localizing candidate objects with open-vocabulary detection. The expert then normalizes the detected boxes across frames, removes duplicated detections with non-maximum suppression, and aggregates the remaining candidates according to the *question-specific counting* target. For *relational or attribute-conditioned counting*, it further uses the available visual or geometric evidence to filter candidates before computing the final count. It returns a structured observation containing the counted target, numerical count, aggregation mode, confidence, and supporting detections.

- **Visual orientation expert** serves as an appearance-grounded orientation specialist that answers questions about the intrinsic facing direction or pose of an object. Given the target object and the original question specified by the planner, it collects the relevant Level-1 visual evidence, such as frames where the object is visible and localized object regions when available. The expert then examines orientation cues including object front/back surfaces, handles, screens, openings, symmetry, and surrounding reference context, and maps the observed pose to the candidate directions or options in the question. Unlike geometric relation experts that compare object positions in 3D space, this expert focuses on the object’s own visual orientation. It returns a structured observation containing the predicted orientation, confidence, and supporting visual evidence.
- **Relative position expert** serves as a 3D relation specialist that answers directional queries between entities, such as left/right, front/back, and cardinal directions. Given the target and reference entities specified by the planner, it first reuses or obtains Level-1 evidence as grounded object boxes, and then queries the Level-2 geometric module to lift these regions into a shared 3D coordinate system. The expert deterministically maps the question to a relation route, such as *object-to-object direction*, *egocentric left/right*, *viewpoint-conditioned direction*, or *cardinal-anchor reasoning*. It then compares the recovered 3D positions under the corresponding reference frame, optionally using camera poses or known direction anchors to calibrate the axes. It returns a structured observation containing the predicted relation or option, confidence, route type, and supporting geometric evidence.
- **Object-centric view expert** serves as a view-aware specialist for questions where the input images are organized around different views of the same target object. Given the target object, labelled viewpoints, and question context specified by the planner, it reuses Level-1 visual evidence from the corresponding object-centric frames and identifies how surrounding objects appear under the specified viewpoint. The expert maps the labelled views, such as front, back, left, and right, to the spatial frame required by the question, and then determines the queried relation from this object-centered coordinate system. It returns a structured observation containing the predicted view-conditioned relation, confidence, and supporting frames.

## C. Details of S-300K

We construct S-300K from SenseNova-800K [5], which is fully disjoint from all evaluation benchmarks used in this work. The construction pipeline consists of three stages: trajectory generation, trajectory filtering, and trajectory decomposition.

**Trajectory generation.** We follow Section 2.2 and sample 100K questions from SenseNova-800K and run zero-shot S-AGENT with GPT-5.4 as the planner to generate tool-use reasoning trajectories. Each trajectory contains the original question, the visual inputs, intermediate planner responses, issued tool calls, returned tool observations, and the final answer produced by the agent.

**Trajectory filtering.** We keep only trajectories whose final answers are valid and correct under the corresponding answer type. Specifically, we discard a trajectory if its execution status is marked as failed, if any unrecovered error occurs, or if no final answer is produced. For multiple-choice questions, we extract the prediction only from the final `<answer> . . . </answer>` field and require the predicted option letter to exactly match the ground-truth option letter. For example, if the ground truth is “B. Northwest”, the trajectory is retained only when the final answer predicts option B. For numeric questions, we parse floating-point values from both the prediction and the ground truth, and compute mean relative accuracy (MRA) with a default threshold of 0.6; the trajectory is retained only if  $\text{MRA} \geq 0.6$ . For free-form text questions, we normalize both the prediction and the ground truth by lowercasing, stripping punctuation and extra whitespace, and then require either exact string match or that the ground-truth answer appears as a substring of the predicted phrase. In short, our SFT data includes only trajectories whose final answers pass answer-type-specific quality checks. We do not use whether a trajectory calls tools as a hard filtering criterion.

**Trajectory decomposition.** After filtering, we decompose each retained trajectory into three complementary supervision formats. First, we construct *final-answer trajectories*, where each original question corresponds to one full trajectory ending in the final answer; these samples train the model to imitate complete S-AGENT reasoning. Second, we construct *turn-level trajectories*, where each VLM planner call is converted into an independent training sample. This reduces excessively long contexts, especially for trajectories involving many images or long tool histories, and exposes the model to intermediate planning decisions. Third, we construct *expert trajectories*, where individual expert or tool calls are converted into specialized sub-samples, such as calls to the metric measurement expert, counting expert, and relative-position expert. A sub-sample is included only when its input is complete, its tool response is available, and the corresponding result can be verified.

**Dataset statistics.** After trajectory generation, filtering, and decomposition, the initial 100K sampled questions yield 292,391 supervised fine-tuning samples. We denote the resulting dataset as S-300K. Table 5 summarizes the data statistics. Starting from 100,000 raw agent traces, 51,596 trajectories pass the quality filtering stage. We keep one final-answer trajectory for each filtered trace, resulting in 51,596 final-answer samples. Trajectory decomposition further produces 154,590 turn-level planner samples and 86,205 nontrivial tool/expert samples. Together, these three categories form S-300K, containing 292,391 supervised fine-tuning samples.

| Data Type                           | Number of Samples |
|-------------------------------------|-------------------|
| Quality-filtered trajectories       | 51,596            |
| Final-answer trajectories           | 51,596            |
| Turn-level trajectories             | 154,590           |
| Nontrivial tool/expert trajectories | 86,205            |
| Total SFT samples in S-300K         | 292,391           |

Table 5 | Statistics of S-300K. The main training set consists of final-answer, turn-level, and nontrivial tool/expert trajectories.

## D. More Experiments

**Results on ReVSI.** Table 6 reports detailed results on ReVSI. S-AGENT achieves an average score of 56.1, ranking third overall and outperforming all open-source general models and

Table 6 | Detailed comparison on the ReVSI [38] leaderboard. ReVSI scores are shown as the main values, and corresponding VSI-Bench scores from the official ReVSI experiments page are shown in gray parentheses when available. We follow the official evaluation dimensions: four numerical question types (object counting, absolute distance, object size, and room size) and three multiple-choice question types (relative distance, relative direction, and route planning). The top-1 / top-2 / top-3 ReVSI results in each column, excluding chance baselines, are highlighted with **deep** , **medium** , and **light** lavender, respectively.

| Model                                               | Frames | Numerical Question |            |           |           | Multiple-Choice Question |           |            | Avg. |
|-----------------------------------------------------|--------|--------------------|------------|-----------|-----------|--------------------------|-----------|------------|------|
|                                                     |        | Obj. Cnt.          | Abs. Dist. | Obj. Size | Room Size | Rel. Dist.               | Rel. Dir. | Route Plan |      |
| <i>Baseline</i>                                     |        |                    |            |           |           |                          |           |            |      |
| Chance (Random)                                     | ALL    | –                  | –          | –         | –         | 23.7                     | 26.8      | 26.0       | –    |
| Chance (Frequency)                                  | ALL    | 52.2               | 40.1       | 17.4      | 20.9      | 25.8                     | 31.9      | 30.2       | 31.4 |
| <i>Proprietary Models (API)</i>                     |        |                    |            |           |           |                          |           |            |      |
| GPT-5.2                                             | 64     | 56.2               | 41.5       | 73.9      | 63.0      | 48.4                     | 34.9      | 38.2       | 50.9 |
| Gemini 3 Flash                                      | 1 FPS  | 65.7               | 53.1       | 77.6      | 52.8      | 64.6                     | 47.9      | 41.8       | 57.6 |
| Gemini 3 Pro                                        | 1 FPS  | 60.1               | 54.7       | 79.3      | 51.9      | 68.1                     | 56.0      | 56.4       | 60.9 |
| <i>Open-Source General Models</i>                   |        |                    |            |           |           |                          |           |            |      |
| Qwen3-VL-8B-Instruct                                | 64     | 40.4               | 52.3       | 69.0      | 45.1      | 57.1                     | 39.5      | 40.5       | 49.1 |
| Qwen3-VL-32B-Instruct                               | 64     | 46.9               | 65.0       | 70.4      | 55.8      | 53.8                     | 34.0      | 47.3       | 53.3 |
| InternVL3.5-8B                                      | 64     | 43.3               | 54.6       | 64.2      | 47.6      | 45.0                     | 36.3      | 44.4       | 47.9 |
| InternVL3.5-38B                                     | 64     | 43.8               | 60.6       | 70.2      | 58.4      | 57.4                     | 45.9      | 42.7       | 54.1 |
| LLaVA-Video-7B-Qwen2                                | 64     | 31.3               | 1.4        | 52.5      | 16.7      | 38.3                     | 33.3      | 38.4       | 30.3 |
| LLaVA-Video-72B-Qwen2                               | 64     | 40.1               | 29.6       | 59.3      | 27.9      | 39.6                     | 24.8      | 43.0       | 37.8 |
| <i>Spatially Specialized Models and Base Models</i> |        |                    |            |           |           |                          |           |            |      |
| Cambrian-S-7B                                       | 128    | 48.4               | 60.5       | 65.5      | 46.7      | 37.1                     | 48.5      | 37.0       | 49.1 |
| Qwen2.5-VL-7B-Instruct                              | 4 FPS  | 36.9               | 15.0       | 49.7      | 29.0      | 31.5                     | 29.5      | 36.7       | 32.6 |
| VST-7B-SFT                                          | 4 FPS  | 35.4               | 52.6       | 67.9      | 47.2      | 49.2                     | 36.9      | 35.4       | 46.4 |
| Qwen2.5-VL-7B-Instruct                              | 32     | 34.3               | 21.7       | 45.5      | 35.1      | 32.6                     | 33.7      | 34.1       | 33.9 |
| SpaceR-7B (SG-RLVR)                                 | 32     | 30.7               | 34.5       | 52.0      | 18.6      | 22.8                     | 34.5      | 20.2       | 30.5 |
| Qwen2.5-VL-3B-Instruct                              | 16     | 18.7               | 15.6       | 16.8      | –         | 33.2                     | 34.3      | –          | 23.7 |
| Spatial-MLLM-4B-135k                                | 16     | 40.7               | 45.3       | 46.8      | –         | 32.3                     | 37.4      | –          | 40.5 |
| Spatial-MLLM-4B-820k                                | 16     | 41.5               | 40.0       | 53.1      | –         | 30.7                     | 39.2      | –          | 40.9 |
| LLaVA-Video-7B-Qwen2                                | 32     | 29.9               | 1.5        | 53.0      | 19.3      | 39.1                     | 33.8      | 38.8       | 30.8 |
| VLM3R-7B                                            | 32     | 41.6               | 61.6       | 64.8      | 52.5      | 46.5                     | 49.5      | 34.1       | 50.1 |
| Ours (S-Agent)                                      | 64     | 34.5               | 45.6       | 62.6      | 53.4      | 63.6                     | 66.4      | 66.1       | 56.1 |

spatially specialized 7B-scale models. The gains are especially pronounced on multiple-choice spatial reasoning tasks: S-AGENT obtains the best results on relative direction and route planning, and ranks third on relative distance. These categories require integrating evidence across frames and viewpoints rather than relying on a single visual impression, which aligns well with the design of stateful evidence accumulation. In contrast, S-AGENT is less competitive on several numerical tasks, such as object counting, absolute distance, and object size estimation. This suggests that while the agentic framework is effective for relational and navigation-oriented reasoning, its numerical accuracy is still constrained by the reliability of the underlying grounding, depth estimation, and metric measurement tools.

**Results on VSR.** Table 7 shows that S-AGENT substantially outperforms existing methods on the VSR subset, achieving particularly large gains in long-video settings. For example, under the 240-minute setting, S-AGENT surpasses the strongest Cambrian-S-7B-LFP baseline by 37.2 percentage points, which we attribute to the introduction and strong performance of our frame-selection tool. However, since the reliability of VSI-SUPER as an indicator of genuine spatial perception has been questioned in recent work [23], we avoid over-interpreting this result and include it mainly as a reference.

Table 7 | Comparison with existing long-video methods on VSI-SUPER. We report results on VSR and VSC under different video durations.

| Eval Setups           | VSR (Duration in Mins.) |             |             |             |             | VSC (Duration in Mins.) |             |             |             |
|-----------------------|-------------------------|-------------|-------------|-------------|-------------|-------------------------|-------------|-------------|-------------|
|                       | 10                      | 30          | 60          | 120         | 240         | 10                      | 30          | 60          | 120         |
| MovieChat             | 18.3                    | 21.7        | 16.7        | 26.7        | 25.6        | 0.0                     | 0.0         | 0.0         | 0.0         |
| Flash-VStream         | 28.3                    | 33.3        | 23.3        | 28.3        | 31.7        | 0.0                     | 0.0         | 0.0         | 0.0         |
| Cambrian-S-7B         | 38.3                    | 35.0        | 6.0         | 0.0         | 0.0         | 0.0                     | 0.0         | 0.0         | 0.0         |
| Cambrian-S-7B-LFP     | 45.0                    | 41.7        | 40.0        | 40.0        | 40.0        | <b>40.6</b>             | <b>42.0</b> | <b>35.0</b> | <b>34.0</b> |
| <b>S-Agent (Ours)</b> | <b>75.0</b>             | <b>55.0</b> | <b>63.3</b> | <b>66.1</b> | <b>77.2</b> | 10.6                    | 4.2         | 0.0         | 0.0         |

## E. Detailed Workflow of S-Agent

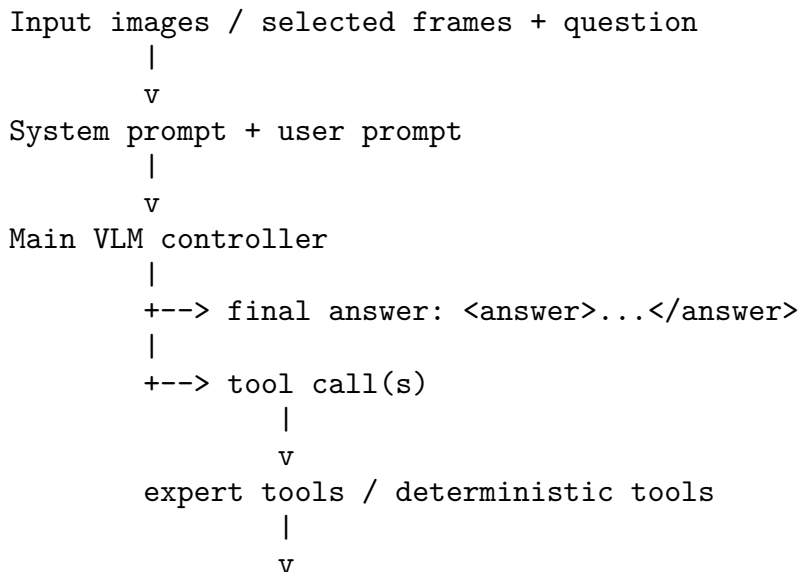
This appendix describes the exact workflow used by S-AGENT. The workflow is shared by both multi-image and video benchmarks. The only difference is the construction of the visual input block: multi-image tasks provide an ordered list of images, while video tasks first convert the video into selected chronological frames and then present these frames as images.

### E.1. Overview of the Prompt Loop

For each sample, S-AGENT runs an iterative controller–tool loop. Let  $q$  be the original question and  $\mathcal{I} = \{I_1, \dots, I_n\}$  be the input images or selected video frames. At iteration  $t$ , the main VLM receives:

- the system prompt, including reasoning and tool-use instructions;
- the user prompt, including image/frame paths and the question;
- a compact memory module summarizing previous tool calls;
- compact tool observations from the immediately previous iteration.

The VLM then either calls one or more tools through native function calling, or outputs the final answer in `<answer>...</answer>` tags.



```

compact observation + memory update
      |
      v
next VLM iteration

```

In our main experiments, S-AGENT uses provider-native function calling. Therefore, the tool schemas are passed to the model API through the `tools` field, rather than being inserted as long JSON text into the prompt.

## E.2. System Prompt

The system prompt contains two parts: a required reasoning protocol and a task-level tool-use guide.

**Reasoning protocol.** Before every tool call and before every final answer, the model is instructed to emit an explicit `<think>...</think>` block. This block is saved for trajectory analysis and SFT export.

You are a helpful assistant that can analyze images and answer questions.

# How to reason (REQUIRED)

Before every tool call AND before every final `<answer>`, you MUST spell out your step-by-step reasoning inside a `<think>...</think>` block placed at the very beginning of your response. The reasoning should cover:

- (a) what you already know from the input image(s) and prior tool observations,
- (b) what information is still missing to answer the question,
- (c) why the specific tool you are about to call (with those arguments), or the final answer you are about to give, follows from (a) and (b).

Always emit the `<think>...</think>` block FIRST in your response text, then either the tool call or the `<answer>`. Never skip the `<think>` block.

**Native tool-use instruction.** When native function calling is enabled, the prompt does not contain XML-style `<tool_call>` examples. Instead, it uses the following instruction:

You have access to a set of tools provided by the runtime.

Call a tool whenever it would help you answer the question; the runtime will execute it and return the result in the next turn. When you have enough information, respond directly without calling any tool.

Remember: the `<think>` block must come FIRST in your response text, before either the function call or the final `<answer>`.

**Tool-use guide.** The controller is then given a task-level routing guide:

# Tool Usage Guide

You have access to specialized vision tools. Use them strategically based on the question type:

### 1. For Counting or Object Finding:

Use `detect_objects_tool` or `vlm_ground_objects` to accurately locate objects across frames. Rely on returned bounding boxes rather than guessing.

2. For Relative Position:  
Call `relative_position_expert_tool` for left/right, front/back, cardinal directions, camera movement, viewpoint, or egocentric spatial questions. Provide images, query\_points, and the exact original question.
3. For Absolute Metric Distance or Object Size:  
Use metric/3D tools. First ground the target objects, then pass the boxes or points into the metric tool, and read the structured metric output instead of estimating distance purely from appearance.

#### # Multi-Step Workflow

1. Carefully analyze the image(s) and the question.
2. Formulate a plan and call the necessary tools.
3. After each tool result, assess whether more information is needed.
4. Continue until there is sufficient evidence.
5. If a tool fails, fallback to visual reasoning rather than guessing randomly.

#### # Final Answer Formulation

The final answer must be enclosed exactly within `<answer></answer>`.

For multiple-choice questions, output only the option letter.

For numerical questions, output only a single number without units.

For text questions, output only the required short phrase.

### E.3. Initial User Prompt for Multi-Image Tasks

For multi-image tasks such as SenseNova and MMSI-Bench, the user prompt lists all input images and then gives the question.

Please analyze the following image(s):

Images to analyze:

- {absolute\_path\_to\_image\_1}
- {absolute\_path\_to\_image\_2}
- ...
- {absolute\_path\_to\_image\_n}

Question:

{original\_question}

Think step by step to analyze the question and provide a detailed answer.

Important Notes:

- You can call tools MULTIPLE times with different parameters to gather comprehensive information.
- After each tool execution, you'll see the results and can decide if you need more information.
- Only provide your final `<answer></answer>` when you have gathered sufficient information.

You MUST output your thinking process in `<think></think>`. Call any of the provided function tools whenever they would help. When you have enough information, output your final choice in `<answer></answer>`. Only put the final answer in `<answer></answer>` tags.

#### E.4. Initial User Prompt for Video Tasks

For video tasks, the raw video is first converted into selected chronological frames. The selected frames may come from uniform sampling, TStar/keyframe selection, or a hybrid strategy. The main controller still receives them as an ordered image sequence.

Here is the sequence of frames selected from the egocentric video in temporal order:

```
Frame 1 (Path: {absolute_path_to_frame_1}): <image>
Frame 2 (Path: {absolute_path_to_frame_2}): <image>
...
Frame n (Path: {absolute_path_to_frame_n}): <image>
```

These are frames of a video.

{original\_question}

{options\_if\_any}

For multiple-choice questions:

Answer with the option's letter from the given choices directly.

For numerical questions:

Answer briefly and directly in one float number.

This video prompt is then inserted as the Question field of the standard S-AGENT user prompt. Thus, after frame selection, the downstream controller workflow is shared with multi-image tasks.

#### E.5. Tool Call Protocol

In native function-calling mode, the VLM does not write explicit JSON tool calls inside the text. Instead, the model response contains:

```
{
  "text": "<think>...</think>",
  "tool_calls": [
    {
      "name": "{tool_name}",
      "arguments": {...}
    }
  ]
}
```

If native function calling is unavailable, S-AGENT can fall back to XML-style calls:

```

<think>...</think>
<tool_call>
{"name": "{tool_name}", "arguments": {"param1": "value1"}}
</tool_call>

```

In the stable GPT/Gemini/Qwen native-tool workflow, the native function-calling path is preferred because it avoids fragile free-form JSON parsing.

## E.6. Continuation Prompt

After tools are executed, S-AGENT starts the next iteration with a continuation prompt. This prompt contains the original question, the previous model response, a compact tool execution summary, original images, generated tool artifacts, memory, and compact observations.

```
=== Multi-Step Analysis: Iteration {t}/{T} ===
```

```
Original Question: {original_question}
```

```
Your Previous Response:
```

```
{last_model_response}
```

```
Tool Execution Summary:
```

```

- {tool_name_iter_k}: Successfully executed
- {tool_name_iter_j}: Failed - {error_message}
...

```

```
Original Images:
```

```

- {original_image_or_frame_1}
- {original_image_or_frame_2}
...

```

```
Generated Images Available for Analysis:
```

```

- {tool_generated_visualization_1}
- {tool_generated_visualization_2}
...

```

```
=== AGENT MEMORY MODULE ===
```

```
[Recent events]
```

```
[Iter 1]: Called {tool_name}. {compact_event_summary}
```

```
[Iter 2]: Called {tool_name}. {compact_event_summary}
```

```
...
```

```
=== Next Steps ===
```

```
You have {remaining_iterations} more iteration(s) available.
```

1. Continue investigating: call tools with corrected or different parameters if more evidence is needed.
2. Provide final answer: if sufficient information is available, output `<think>...</think>` and then `<answer>...</answer>`.

Please continue:

```
Tool Observations:
[Observation from {tool_name}]:
{compact_tool_observation}
```

Only compact observations are fed back to the VLM. Large raw outputs, long detection tables, and raw point clouds are saved in the trajectory files but are not fully inserted into the next prompt.

### E.7. Tool Observation Compression

Tool outputs are converted into short text observations before being injected into the next VLM turn. S-AGENT prioritizes the fields `summary` and `result`. If those fields exist, they are inserted directly:

```
[Observation from metric_measurement_expert_tool]:
object_size=1.03 m; confidence=high
```

If no summary exists, S-AGENT builds a compact JSON preview and removes large visual fields such as:

```
vis_path, vis_paths, output_path, processed_images, all_hits_per_target
```

This keeps the next prompt short while preserving the complete raw artifact in the saved trajectory directory.

### E.8. Memory Prompt

S-AGENT maintains a lightweight memory module. The memory is not a full transcript. It stores compact per-round facts such as which tool was called, whether it succeeded, and what high-level evidence it produced.

A typical memory block looks like:

```
=== AGENT MEMORY MODULE ===
[Recent events]
[Iter 1]: Called vlm_ground_objects. Grounded 2/4 targets.
[Iter 2]: Called metric_measurement_expert_tool successfully.
```

This memory block helps the VLM avoid repeating the same failed call and encourages it to synthesize current evidence.

### E.9. Expert-Level Prompt Surfaces

Some tools are deterministic wrappers and do not call another VLM. Others are expert tools with their own compact prompts.

**VLM grounding.** The VLM grounding tool uses the VLM only for frame selection, not box prediction. The prompt is:

Determine which of the listed target objects are CLEARLY VISIBLE in this image.

Targets: {targets}

For EACH target, output one JSON object with:

```
"label": "<exact target string>"
"visible": <true | false>
"confidence": <float 0-1>
```

DO NOT output bounding boxes. Bounding box prediction is handled by a downstream detector. Your only job is to judge whether each target is present in this image.

Output ONLY a JSON array, no prose, no markdown fences.

The selected best frame is then passed to GroundingDINO to obtain precise bounding boxes.

**Relative position expert.** The relative position expert receives a BEV map, selected frames, 3D points, deterministic geometric facts, and the original question. Its prompt is:

You are a specialized 3D spatial reasoning expert.  
You are provided with a Global Bird's-Eye View (BEV) Radar Map and relevant camera frames.

V5 route analysis:  
{route\_json}

Here are the precise 3D coordinates extracted by the metric depth sensor:  
{points\_info}

Computed geometric facts:  
{geometric\_facts}

Query-specific deterministic candidates:  
{candidate\_section}

Question: "{original\_question}"

If the question is multiple-choice:  
The allowed options are:

- A. ...
- B. ...
- C. ...
- D. ...

Your <result> MUST contain only the single option letter.

Rules:

1. Trust high-confidence deterministic candidates when they exactly match the question.
2. Do not use object center points to answer intrinsic object-facing questions.

3. For large planes or surfaces such as walls, ceilings, windows, doors, and mirrors, center-point geometry can be noisy.
4. Put reasoning inside `<think>...</think>`, then put only the final answer inside `<result>...</result>`.

The main agent only sees the compact result, for example:

Relative-position expert conclusion: C

**Visual orientation expert.** This expert is used for intrinsic object orientation, such as front/back/side of an object. Its prompt explicitly separates local object orientation from egocentric or compass-frame mapping:

You are a visual orientation expert, but you must abstain when the images do not provide enough evidence or when the reference-frame mapping is not solved.

Task type: two-stage visual orientation.

Target object: {target\_object}

Reference context: {reference\_context}

Stage 1: visual evidence for the target itself.

- Decide whether the target is visible.
- Look for front/back/side/surface-normal cues: doors, handles, panels, openings, mirrors, grilles, sink basins, wall mounting.
- Do not infer a direction from object location alone.

Stage 2: reference-frame mapping.

- Parse whether the question defines a separate frame, such as facing an object, sitting/standing, interaction, window view, or compass direction.
- Only choose an option if visual evidence plus reference context are both sufficient.
- If mapping is not solved, return uncertain.

Question:

{original\_question}

Options:

- A. ...
- B. ...
- C. ...
- D. ...

Return compact JSON inside `<result>...</result>`:

```
{
  "target_visible": true/false,
  "front_evidence": ["..."],
  "local_orientation": "front/back/left/right/normal/unknown",
  "reference_context": "...",
  "mapping_evidence": "...",
  "confidence": "high/medium/low",
```

```

    "abstain_reason": "...",
    "result": "A/B/C/D/uncertain"
}

```

**Object-centric view expert.** This expert handles questions where multiple images show the same object from labeled views, such as front/left/back/right.

You are an object-centric multi-view spatial expert.

The question states that the images show one target object from labelled views:  
{view\_order}

Requested image number: {image\_number}

Requested side of the target in that requested image: {requested\_side}

Object-centric side implied by that viewer-side request: {mapped\_side}

Labelled image that shows the implied object side: image {mapped\_image}

Your task:

1. Treat the listed front/left/back/right labels as object-centric view labels.
2. Convert the requested viewer-side into the corresponding object-centric side.
3. Inspect the labelled image for the implied object side.
4. Choose the option that best describes the surroundings/object on that side.
5. If the side mapping or visual identification is ambiguous, return uncertain.

Question:

{original\_question}

Options:

- A. ...
- B. ...
- C. ...
- D. ...

Return compact JSON inside <result>...</result>:

```

{
  "target_visible_in_requested_image": true/false,
  "mapped_object_side": "...",
  "mapped_image_number": "...",
  "requested_side_evidence": "...",
  "rejected_distractors": ["..."],
  "confidence": "high/medium/low",
  "result": "A/B/C/D/uncertain"
}

```

## E.10. Deterministic Tool Interfaces

The counting and metric experts are mainly deterministic wrappers. Their inputs are structured arguments rather than long sub-prompts.

**Counting expert input.**

```
counting_expert_tool(
    images = [image_or_frame_paths],
    question = exact_original_question,
    target_object = optional_extracted_target
)
```

It extracts the counted object, runs open-vocabulary detection, deduplicates boxes with NMS, and aggregates conservatively across frames.

#### **Metric measurement expert input.**

```
metric_measurement_expert_tool(
    images = [image_or_frame_paths],
    question = exact_original_question,
    query_points = optional_grounded_boxes_or_points,
    target_objects = optional_target_labels
)
```

It classifies the measurement route, grounds targets if needed, invokes metric 3D depth, and returns compact numeric evidence.

### **E.11. Controller-Side Recovery and Preflight**

Besides the visible prompt, S-AGENT has a controller-side safety layer before executing tools.

**Grounding recovery.** If a relative-position question requires entities that have not yet been grounded, S-AGENT may inject a small number of generic detection calls. This is answer-agnostic: it only asks for missing named entities and never reads the ground-truth answer.

**Relative-position preflight.** Before running the relative-position expert, S-AGENT checks whether the required query points are complete. If key entities are missing, it returns an uncertain result instead of running geometry on incomplete boxes.

**Final answer guard.** If the relative-position expert returns a high-confidence option letter, S-AGENT can force the final answer to match that expert conclusion:

```
<think>High-confidence tool result from relative_position_expert_tool
was used as the final answer.</think>
<answer>{expert_option}</answer>
```

This prevents the main VLM from accidentally changing a high-confidence expert result during final synthesis.

### **E.12. Final Answer Extraction**

The evaluator only extracts answers from:

```
<answer>...</answer>
```

For multiple-choice tasks, the content must be a single option letter. For numeric tasks, the content must be a single integer or float without units. Tool reasoning, memory, and <think> blocks are never used for answer extraction.

## F. Limitations

Despite its strong performance, S-AGENT has several limitations. S-AGENT still depends on the reliability of external spatial tools, such as object grounding, depth estimation, and 3D lifting, so errors in these tools may propagate to the final answer. Its multi-step tool-use process also introduces higher inference cost than single-shot VLM prediction. In addition, the framework requires a planner with sufficient tool-use ability; as shown in our experiments, directly equipping a smaller VLM with tools may bring limited gains or even hurt performance. Future work will explore more efficient tool use and more robust spatial memory in interactive embodied settings.